Creating LLM Concept Sets

Joel N. Swerdel

2026-10-13

Contents

1 Introduction **2**

2 Overview of Process **2**
2.1 Selecting the health condition and the SNOMED concept 2
2.2 Gather your database information 2
2.3 Create your LLM connection information 2

3 Output **3**
3.1 Cache files 3

1 Introduction

The `Phenelope` package enables evaluating the creating of a health condition concept set. The output of the process is a JSON expression that can be imported directly into ATLAS to a concept set.

This vignette describes how to run the Phenelope process from start to end in the `Phenelope` package.

2 Overview of Process

There is only a small amount of information needed to create a concept set:

1. The name of a health condition and a SNOMED concept Id that is close to the health condition.
2. Database information to access the database and the associated vocabulary.
3. A LLM connection object.

Each of these steps is described in detail below. For this vignette, we will describe the request for a concept set for type 2 diabetes mellitus (T2DM).

2.1 Selecting the health condition and the SNOMED concept

The health condtion name to be used for the process would be something like "Type 2 Diabetes Mellitus". After examining the database, an appropriate SNOMED concept id is 201826 for "Type 2 diabetes mellitus".

Quick Tip: Phenelope uses PHOEBE to determine possible other concepts that are not direct children of the selected concept. You could, for instance select, 201820 for "Diabetes mellitus", the parent of 201826. This will expand the number of concepts that the LLM will test. Depending on your health condition, this may or may not be desirable. The only cost to using a parent concept is the number of concepts that are eventually tested, which could increase the amount of time to create the concept set.

2.2 Gather your database information

The information you will need to gather for connecting to the database is:

1. The database connection string as is typically used in HADES packages.
2. The database schema where the common data model is located (commonly used in "cdmDatabaseSchema" in HADES packages)

Below is an example for creating a database connection string:

```
# create database connection details
connectionDetails <- DatabaseConnector::createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/ohdsi",
  user = "joe",
  password = "supersecret"
)
```

2.3 Create your LLM connection information

The information you will need is documented in the ellmer package. See, for example, the ellmer function chat_azure_openai. Required fields are LLM dependent, but, as an example, you may need: * endpoint * api_version * model * credentials

For example:

```
# create LLM client for ellmer
llmClient <- ellmer::chat_azure_openai(
  endpoint = gsub("/openai/deployments.*", "", keyring::key_get("genai_gpto3_endpoint")),
```

```
  api_version = "2023-03-15-preview",
  model = "o3",
  credentials = function() keyring::key_get("genai_api_gpt4_key")
)
```

The example above creates an ellmer object that will connect to an Azure Open AI instance, version o3.

The recommended input is both a name and a clinical description of the idea for which we want to create a concept set:

```
targetName <- "Acute liver failure"
targetClinicalDefinition <- "Acute liver failure is a rare but life-threatening syndrome characterized
```

Once you have the necessary information, you are ready to create you concept set by calling the createConceptSet() function as shown below:

```
# run the
finalSet <- Penelope::createConceptSet(
  name = targetName
  clinicalDefinition = targetClinicalDefinition,
  llmClient = llmClient,
  connectionDetails = connectionDetails,
  vocabDatabaseSchema = "yourCDMSchema",
  cacheFolder = "c:/llmConceptSets/type2Dm"
)
```

Additionally, it is possible to provide one or more seed concepts - concepts that are close the concept set target - through the `seedConceptIds` argument.

3 Output

The function returns a JSON expression of the concept set.

3.1 Cache files

If you provided a cache folder, you will find in that folder:

1. **ConceptSetExpression.json**: the same JSON expression returned by the function
2. **FinalConcepts.csv**: A csv file with the full set of evaluated concepts and their final status.

As well as intermediary files that can be used to quickly rerun parts or all of the process.